

Public contract: JSON surfaces and exit codes

This document declares the stability status of every machine-readable surface BackupSage exposes, and how the golden fixtures that freeze them work. The compatibility window (how long these promises hold, across which versions) is tracked separately in issue #57.

Golden fixtures

Every surface below is frozen as a committed fixture under `tests/fixtures/contract/` and compared on every test run by `tests/contract.rs` — ten fixtures: one per surface plus completed-with-skips variants of `search --all --json` and `dedup --json` (populated `skipped[]` / `archives_offline`), plus `dedup_chain.json`, which pins the keeper-star fields (#9) at non-degenerate values — a real transitive-only member with `actionable: false` and `hamming_to_keep > 3` — plus two more non-degenerate content-mode corpora (#71): `search_only.json` (a search-only archive's `mode` field on `search --all --json`) and `metadata_only_skips.json` (a metadata-only archive's worded skip on `dedup --json`, alongside a normal duplicate pair so `summary.groups` stays non-zero too):

```
cargo test --test contract          # verify against the frozen contract
BACKUPSAGE_BLESS=1 cargo test --test contract  # regenerate deliberately
```

Comparison is semantic JSON equality after normalizing the only run-varying values (temp-directory prefixes become `<TMP>`, the wall-clock `indexed_unix` becomes `"<TS>"`). Any field rename, removal, addition, or type change fails the suite. **The contract is additive-only**: a re-bless whose diff adds fields may ship; a re-bless whose diff renames or removes fields is a breaking change and must not. Fixture generation only reads archives and writes to fresh temp paths — BackupSage never rewrites or deletes from an archive.

JSON surfaces

Surface	Shape	Status
dedup --json	Typed structs in <code>src/report.rs</code> , top-level <code>"version": 1</code>	Stable. Versioned, additive-only, consumed by scripts and the future web UI. v1.0.2 (#9) added <code>Member.actionable</code> , <code>Group.review_only_bytes</code> , <code>Summary.transitive_only_files/review_only_bytes</code> , <code>params.actionable_rule</code> — additive, still version 1. <code>reclaimable_bytes/duplicate_files</code> now count only keeper-star-safe members (correctness fix: previously inflated by transitive-only members whose distance to the keeper exceeds the threshold). v1.0.2 (#71) also added <code>ReportArchive.content_mode</code> and a metadata-only case in <code>summary.skipped_archives</code> (alongside the existing <code>v2-limited</code> case) — additive, still version 1. A metadata-only-only master now exits 2 where it previously exited 0 with an empty, unexplained report.
search --json	<code>{query, hits[], truncated, mode}</code> ; hits carry <code>path</code> , <code>matches</code> , <code>snippet</code> , and <code>path_bytes</code> (hex) only for non-UTF-8 paths	Frozen-as-observed. No version field yet; fixture-protected; changes must be additive. v1.0.2 (#70) added top-level <code>mode</code> (<code>full/search-only/metadata-only</code>) — additive. On a search-only index <code>matches</code> is <code>null</code> : contentless FTS cannot run <code>highlight()</code> , and <code>snippet</code> is absent for the same reason. Full-mode output is byte-identical to earlier v1.x.
search --all --json	<code>{archives[{archive, truncated, mode, hits[]}], skipped[{archive, reason}]}</code>	Frozen-as-observed. Same rules as <code>search --json</code> , including <code>matches: null</code> from a search-only child. A metadata-only child is never searched: it appears in <code>skipped</code> with a worded reason, which is exit 2. v1.0.2 (#71) added the per-archive <code>mode</code> field — additive, present on every successfully-searched archive. <code>--snippets</code> against a search-only archive now also prints a stderr note, matching the single-archive path's existing note.
master list --json	Array of <code>{archive_id, label, source, source_type, db_path, schema_version, files, completed, status, indexed_unix, content_mode}</code>	Frozen-as-observed. Same rules. v1.0.2 (#71) added <code>content_mode</code> — additive.
master verify --json	Array of <code>{archive_id, label, status, source}</code>	Frozen-as-observed. Same rules.
Plan document	Typed structs in <code>src/plan.rs</code> , top-level <code>"plan_schema_version":</code>	Stable, versioned, BYTE-EXACT — stricter than every surface above: key order (struct declaration order), whitespace (<code>to_string_pretty</code> , two-space indent, one trailing newline) and number formatting

Surface	Shape	Status
	1, schema at docs/schema/plan-v1.schema.json	are all part of the contract, not incidental to it. plan_schema_version is independent of index_schema_version and of the dedup report's version: 1. Compared by raw byte equality in src/plan.rs's own #[cfg(test)] mod contract, not by tests/contract.rs's semantic serde_json::Value comparator, which cannot see a key-order or whitespace change. See ADR 0006 (#75).

"Frozen-as-observed" means: the shape carries no version field today, but the golden fixtures pin it exactly, so any non-additive change is caught in CI. Giving these surfaces a version field is itself an additive change and may happen in a later release.

Known residual gaps (fields the fixtures pin only at a degenerate value, so a type change to them would not be caught): `hardlink_of` is null on every corpus member (no hardlink entries yet), and `archives_incomplete` appears only as an empty array. `sparse` stays pinned false in dedup output by design — sparse rows are excluded from dedup candidates at the SQL level — so its evidence lives elsewhere: the #64 differential corpus (tests/fixtures/sparse/, four GNU-tar-built dialect archives, frozen-as-observed) proves old-GNU logical size/bytes/BLAKE3 against GNU tar across tar/gz/zstd, pins the PAX name-only shape, and drives every malformed-map case to a loud abort in tests/sparse.rs. The element shape of `summary.skipped_archives` is additionally pinned by an inline assertion in tests/cli.rs (v2-limited flow). Field renames and removals of all of these are still caught — the keys themselves are in the fixtures.

All other commands (`index`, `top`, `inspect`, `master add/sync/rm`) produce human-oriented text only — **unstable**, no machine-readable promise.

Exit codes

Documented in `src/main.rs` and executed as a per-subcommand matrix in `tests/contract.rs::exit_code_matrix`:

Code	Meaning
0	Completed cleanly. Includes "no results" and "no duplicates".
1	Error: bad input, missing file/index/master, unknown key or path.
2	Completed with skips : <code>dedup</code> skipped offline/incomplete/v2-limited archives; <code>search --all</code> skipped offline or unreadable archives (v2-limited archives are fully searchable and do not cause exit 2); or <code>master verify</code> found any non-ok archive.

Caveat frozen as observed behavior: **clap usage errors also exit 2** (unknown flag, missing subcommand). They are distinguishable from completed-with-skips — usage errors print help/usage text to stderr and perform no work. Scripts distinguishing the two should treat exit 2 with a usage message on stderr as an invocation bug, not a skip report.